

Python Interview Guide

Core OOP Concepts: Abstraction & Polymorphism

30 Q&A MASTER SHEET

Q1 `Payment.process(self, amount)` is overridden as `UPI.process(self, amount, pin)`. What is the best assessment?

Assessment: Violates the **Liskov Substitution Principle (LSP)**. Adding mandatory parameters (`pin`) breaks callers expecting the base interface signature `process(amount)`.

Best Fix: Accept details in `__init__` or use optional default parameters:

```
class Payment:
    def process(self, amount: float): pass

class UPI(Payment):
    def __init__(self, pin: str): self.pin = pin
    def process(self, amount: float): print(f"Paid {amount} with PIN")
```

Q2 A parent declares `@abstractmethod run()`. A child implements it, and a grandchild redeclares it as `@abstractmethod`. What happens?

The grandchild class becomes abstract again. Attempting to instantiate it raises a `TypeError` at runtime:

```
from abc import ABC, abstractmethod

class Parent(ABC):
    @abstractmethod
    def run(self): pass

class Child(Parent):
    def run(self): print("Running")

class GrandChild(Child):
    @abstractmethod
    def run(self): pass

g = GrandChild() # TypeError: Can't instantiate abstract class GrandChild
```

Q3 `B(A)` and `C(A)` define `show()` printing their letter + `super().show()`. `A.show()` prints "A". `D(B, C)` has no body. What does `D().show()` print?

Output: B C A

Explanation: Python uses **C3 Linearization** for Method Resolution Order (MRO). For class `D`, MRO is `D → B → C → A → object`. Calling `super().show()` inside `B` invokes `C.show()`, not `A.show()`.

Q4 An abstract `calculate_fee()` is implemented by one child returning `float` and another returning `str`. What is the best fix?

Fix: Enforce a single consistent return type hint in the base class interface or return a standardized data object so polymorphic callers do not need conditional type handling.

```
class FeeCalculator(ABC):
    @abstractmethod
    def calculate_fee(self) -> float: pass
```

Q5 A developer says, "Abstraction means hiding everything, so I'll create one huge abstract class." What is the strongest challenge?

Challenge: Violates the **Interface Segregation Principle (ISP)**. Monolithic classes force subclasses to implement unused abstract methods, creating tight coupling and fragile class hierarchies.

Q6 A notification system supports Email, SMS, WhatsApp, Push. Which design best keeps business logic channel-agnostic?

Define an abstract base interface `NotificationChannel` and program business logic against it:

```
class NotificationChannel(ABC):
    @abstractmethod
    def send(self, recipient: str, message: str) -> None: pass

class EmailChannel(NotificationChannel):
    def send(self, recipient: str, message: str): print(f"Emailing {recipient}")

class NotificationService:
    def __init__(self, channel: NotificationChannel):
        self.channel = channel  # Channel-agnostic
```

Q7 How do you let a document-processing system support a new file type without modifying existing processing logic?

Apply the **Open/Closed Principle (OCP)** using a Strategy pattern or abstract interface. Define a common `DocumentProcessor` and add new file handling classes without touching core processing pipeline code.

Q8 In a ride-booking system, Bike/Car/Auto/PremiumCar each calculate fares differently. Where does fare logic belong?

Fare logic belongs encapsulated inside each concrete vehicle subclass, overriding a common abstract method `calculate_fare(distance, duration)` defined on the parent `Vehicle` class.

Q9 A developer says if-elif on `payment_type` is simpler than polymorphism. When might they be right?

When domain logic is simple, static, unlikely to change or expand, and localized to a single place. In small, fixed operations, full OOP polymorphism can introduce unnecessary boilerplate.

Q10 Your team wants to replace payment implementation without touching checkout. What boundary do you create?

Create an abstract boundary interface (e.g., `PaymentGateway`). `CheckoutService` interacts solely with this abstract interface, decoupling checkout flow from implementation details.

Q11 `Storage.save()` has children `MySQLStorage`, `MongoStorage`, `FileStorage`. Which is the best-defended choice?

Choice: Define `Storage.save()` as an abstract method within a repository contract.

Reason: Protects callers against database implementation specifics and enables easy unit test mocking.

Q12 An ML app has `LinearRegression`, `DecisionTree`, etc., each with `train()` and `predict()`. How do you let the pipeline work with third-party models?

Use **Duck Typing** or Python's `typing.Protocol` (Structural Subtyping). Models do not need to inherit from a common base class as long as they implement required methods:

```
from typing import Protocol, Any

class MLModel(Protocol):
    def train(self, X: Any, y: Any) -> None: ...
    def predict(self, X: Any) -> Any: ...
```

Q13 `Payment.process(payment_type)` branches on a string, and `UPIPayment/CardPayment` are added. Why does this defeat polymorphism?

It violates the **Open/Closed Principle**. Adding new payment methods forces modifications to central conditional code rather than cleanly extending behavior via subclasses.

Q14 A legacy codebase uses `type(obj) == X` and `isinstance(obj, X)` everywhere. What does this most likely indicate?

Indicates **procedural control flow** disguised as OOP code. Hardcodes tight dependencies on specific concrete classes, destroying duck typing and extensibility.

Q15 A plugin system needs `initialize()`, `execute()`, and `shutdown()` from third parties. Which architecture works best?

An Abstract Base Class interface or Protocol contract defining exact operational lifecycle steps for third-party extensions.

Q16 Why would an ABC deliberately mix concrete and abstract methods?

To implement the **Template Method Pattern**. Concrete methods define the reusable execution workflow skeleton while delegating specific custom steps to abstract methods implemented by subclasses.

```
class Pipeline(ABC):
    def run(self): # Concrete template method
        self.extract()
        self.transform()

    @abstractmethod
    def extract(self): pass
```

Q17 With no declared interface, what decides whether an object can participate in polymorphism in Python?

Duck Typing. Python checks whether required method names and attributes exist dynamically at execution time ("if it walks and quacks like a duck, it's a duck").

Q18 When is an ABC preferable to pure duck typing in production?

When strict interface contracts, early validation at instantiation time (failing early before deep execution), or clear architectural standards are required across large developer teams.

Q19 What is the main problem with an ABC that has 15-20 abstract methods?

Violates the **Interface Segregation Principle (ISP)**. Creates monolithic class structures where child classes are forced to write dummy implementations for methods they do not need.

Q20 What does `isinstance(obj, P)` check when `P` is a `@runtime_checkable` Protocol?

It checks for the physical presence of required attributes/methods on `obj` using standard internal `hasattr` checks. It does *not* validate argument counts or type signatures.

Q21 After Payment.register(LegacyPay), what is true?

LegacyPay becomes a **Virtual Subclass** of Payment. `isinstance(LegacyPay(), Payment)` returns True, but LegacyPay inherits no attributes or methods from Payment.

Q22 When does Python enforce unimplemented abstract methods?

At **instantiation time** (when invoking `MyClass()`), not during module import or class definition time.

Q23 What is the correct way to declare an abstract property?

Stack `@property` on top of `@abstractmethod`:

```
class Entity(ABC):
    @property
    @abstractmethod
    def id(self) -> str: pass
```

Q24 Multiple mixins and a base class are combined with multiple inheritance. Safest `__init__` practice?

Always call `super().__init__(**kwargs)` and forward unhandled keyword parameters down the Method Resolution Order (MRO) chain.

Q25 What does `__init_subclass__` give a plugin system, and what is the catch?

Benefit: Automatically hooks and registers child classes upon import without metaclass complexity.

Catch: Executes at module import/definition time; misconfigured classes fail immediately upon load.

Q26 An @abstractmethod has a body. Is that meaningful?

Yes. It provides default reusable logic that child classes can explicitly invoke via `super().method()` while still forcing subclasses to implement an override.

Q27 Square subclasses Rectangle, with width/height setters kept in sync. What's the issue?

Violates the **Liskov Substitution Principle (LSP)**. Modifying width on a Square alters height unexpectedly, breaking code that relies on standard `Rectangle` behaviors.

Q28 Key difference between raise NotImplementedError and @abstractmethod?

Feature	raise NotImplementedError	@abstractmethod
Enforcement	Fails at runtime when called	Fails at instantiation time
Requirement	Voluntary override	Mandatory override before object creation

Q29 An interface defines fetch(). Some implementations are async def fetch(), others sync. What's the problem?

Breaks polymorphic execution calling semantics. Invoking an `async def` method yields a coroutine object that requires `await`, causing runtime errors in synchronous execution contexts.

Q30 When is functools.singledispatch preferable to method-based polymorphism?

When extending behavior or adding dynamic dispatch for classes/types you do not control or cannot modify (e.g., standard built-ins or third-party library classes):

```
from functools import singledispatch

@singledispatch
def process(data): raise NotImplementedError()

@process.register(int)
def _(data: int): return data * 2
```